Data Cleaning

python.exposed
Learn Python to Automate, Analyze, Accelerate.

Lesmateriaal

- Via GitHub account:
- git clone https://github.com/oclaerbout/syntra_data_scientist.git
- © 2024 Olivier Claerbout
- *Dit materiaal mag niet worden gekopieerd, verspreid, gepubliceerd of anderszins gereproduceerd zonder uitdrukkelijke schriftelijke toestemming van de auteur. Dit geldt eveneens voor de Jupyter Notebooks!*

Basisprincipes van data cleaning

SYNTRA ANTWERPEN & VLAAMS-BRABANT

Data cleaning

- Data Cleaning is het opschonen van datasets door fouten, inconsistenties en irrelevante gegevens te corrigeren.
- Het is een cruciale eerste stap in data-analyse om de kwaliteit van modellen te waarborgen.
- Het doel van Data Cleaning is:
  - Het verbeteren van de betrouwbaarheid en nauwkeurigheid van analyses.
  - Verminderen van ruis in data om betere inzichten te verkrijgen.

SYNTRA ANTWERPEN & VLAAMS-BRABANT

Wat maakt data 'dirty'?

- Ontbrekende waarden
- Uitschieters
- Dubbele waarden
- Foutieve data
- Inconsistenties
  - In eenheden (vb reporting in km, mijl, …)
  - In stijl (mm-dd-yyyy, mm/dd/yyyy, …)
  - In filetype (.jpg, .png, …)

Dergelijke zaken zorgen ervoor dat je analyse incorrect kan zijn, of je code zelfs niet kan runnen…

Leer je dataset kennen

- Van waar afkomstig? *Zit er bias in de bron? Betrouwbare data?*
- Aantal 'features'? *Typisch zijn dit je kolommen…*
- Categorieën binnen features? *Vb man-vrouw,…*
- Bekijk de samenvattende statistiek per kolom *(gemiddelde, mediaan, variantie, standaardafwijking)*
- Plot indien mogelijk de variabelen en de variabelenparen. *Check naar scheve verdelingen, uitschieters, trends, correlaties,…*
- Indien data uit db: *kijk naar de relaties en begrijp de relatie tussen tabellen*
- Gebruik indien mogelijk de profiling feature van pandas om een rapport inzake je dataset te genereren

Learn Python to Automate, Analyze, Accelerate.

Outliers

Outliers (uitbijters)

- Outliers zijn waarden die significant afwijken van andere observaties in de dataset.
- Ze kunnen het resultaat zijn van fouten in gegevensinvoer of natuurlijke variatie.
- Sommige veronderstelde outliers zijn eigenlijk waardevolle data
  - Voorbeeld: reactie beurzen op COVID-19,...
- Andere outliers zijn fouten of irrelevante datapunten die weg mogen
- Detectie van outliers:
  - Visueel: Boxplots, scatterplots.
  - Statistisch: Gebruik van z-scores of percentielen.

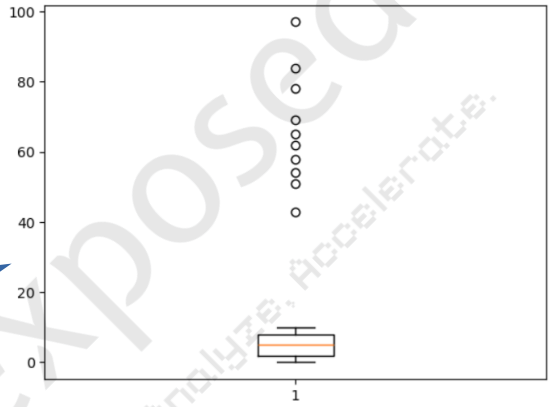
Detectie van Outliers

- Detectie van outliers:
  - Visueel: Boxplots, scatterplots.
  - Statistisch: Gebruik van z-scores of percentielen.

```python
import matplotlib.pyplot as plt

plt.boxplot(data)
```

Via boxplot



SYNTRA ANTWERPEN & VLAAMS-BRABANT

Learn Python to Automate, Analyze, Accelerate.
Python.exposed


```python
import numpy as np
import pandas as pd

# Generate some sample data
np.random.seed(0)
data = np.random.randint(low=0, high=11, size=1000)

# Add some outliers
outlierdata = np.random.randint(low=40, high=100, size=10)
print(f"Outliers added: {outlierdata}")
# Choose random indexes
outlier_indexes = np.random.randint(low=0, high=999, size=10)

# Zip data together
outlierpoints = zip(outlier_indexes, outlierdata)

for point in outlierpoints:
    data[point[0]] = point[1]
```

Via Z-scores

```python
# Calculate Z-scores
z_scores = (data - np.mean(data)) / np.std(data)

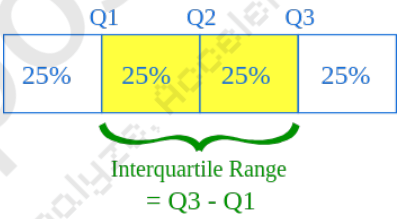
# Identify outliers based on Z-score threshold (e.g., 3)
threshold = 3
outliers = np.where(np.abs(z_scores) > threshold)[0]

print("Outliers identified using Z-score method:")
print(data[outliers])
```

```
Outliers added: [62 97 69 78 51 54 58 65 84 43]
Outliers identified using Z-score method:
[84 78 62 97 58 43 51 65 69 54]
```

Detectie outliers via percentielen

- Wat is de Interkwartielafstand (IQR)?
  - IQR staat voor Interquartile Range of interkwartielafstand.
  - Het meet de spreiding van het middelste 50% van de data.
  - Formule: IQR = Q3 - Q1
    - *Q1 (25e percentiel): Waarde waar 25% van de data onder ligt.*
    - *Q3 (75e percentiel): Waarde waar 75% van de data onder ligt.*



python.exposed
learn python to automate, analyse, accelerate.

Detectie outliers via percentielen

Via IQR

```python
# Calculate quartiles and IQR
q1 = np.percentile(data, 25)
q3 = np.percentile(data, 75)
iqr = q3 - q1

# Identify outliers based on IQR
lower_bound = q1 - (1.5 * iqr)
upper_bound = q3 + (1.5 * iqr)

outliers = np.where((data < lower_bound) | (data > upper_bound))[0]
print("Outliers identified using IQR method:")
print(data[outliers])
```

```
Outliers identified using IQR method:
[84 78 62 97 58 43 51 65 69 54]
```

Behandeling van Outliers

- Verwijderen: Geschikt als het gaat om datapunten die fouten zijn, indien de fouten niet kunnen worden opgelost.
- Outliers kunnen worden vervangen door een minder extreme waarde waarbij de vorm van de distributie wordt behouden
- Capping: alles boven of onder een bepaalde treshold wordt vervangen door een specifieke waarde
- Transformatie: Logaritmische of worteltransformaties om de impact van outliers te verminderen.
- Imputeren: Vervangen door mediaan of gemiddelde van de kolom

SYNTRA ANTWERPEN & VLAAMS-BRABANT

Behandeling van Outliers - capping

```
# Define the lower and upper thresholds for capping (Here I used the 5th and 95th percentiles)
lower_threshold = np.percentile(data, 0.05)
upper_threshold = np.percentile(data, 0.95)

# Cap
capped = np.where((data > lower_threshold) | (data < upper_threshold))[0]
print("Remaining datapoints after capping:")
print(len(data[capped]))
```

```
Remaining datapoints after capping:
912
```


SYNTRA ANTWERPEN & VLAAMS-BRABANT

Transformatie van Features

- Aanpassen van de schaal of distributie van variabelen om de prestaties van modellen te verbeteren.
- Veelgebruikte transformaties:
  - Logtransformatie: Voor scheve verdelingen.

```
df['log_kolom'] = df['kolom'].apply(lambda x: np.log(x + 1))
```

Om te vermijden log(0) te nemen

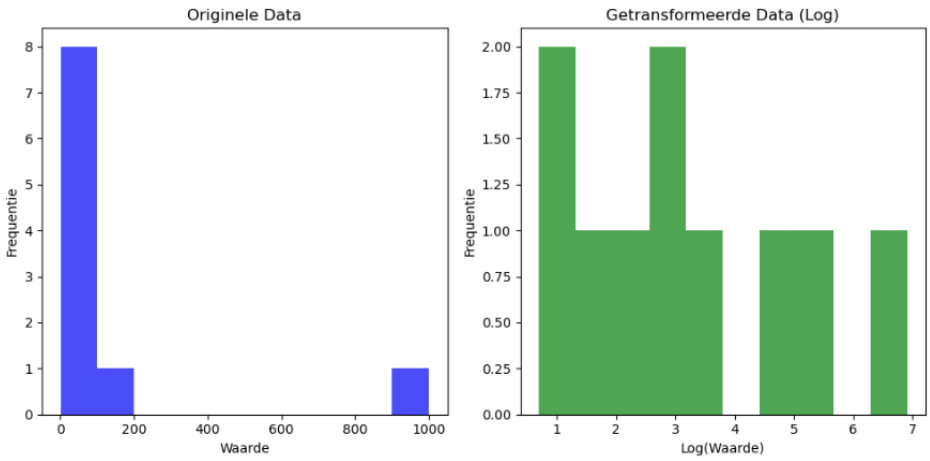
  - Standaardisatie: Z-score transformatie om gemiddelde 0 en standaarddeviatie 1 te krijgen.

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
df['kolom_scaled'] = scaler.fit_transform(df[['kolom']])
```


SYNTRA ANTWERPEN & VLAAMS-BRABANT

Transformatie van Features

- De getransformeerde data heeft een meer gebalanceerde verdeling en outliers hebben minder invloed.



Transformatie van Features

- Veelgebruikte transformaties:
  - Normalisatie: Omzetten naar een bereik van 0-1 met Min-Max scaling.

```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
df['kolom_norm'] = scaler.fit_transform(df[['kolom']])
```


python.exposed Learn Python to Automate, Analyze, Accelerate.

Ontbrekende data

Ontbrekende Waarden in Pandas

- Gegevens die niet beschikbaar zijn in een dataset.
- Vaak weergegeven als NaN (Not a Number).
- Kunnen analyses vertekenen.
- Invloed op de nauwkeurigheid van modellen.

python.exposed
Learn Python to Automate, Analyze, Accelerate.

Soorten Ontbrekende Waarden

- MCAR (Missing Completely At Random):
  - Ontbrekende waarden hebben geen relatie met andere data.
  - Voorbeeld: Een uitval in gegevensverzameling.
- MAR (Missing At Random):
  - Ontbrekende waarden zijn afhankelijk van bekende gegevens.
  - Voorbeeld: Leeftijd ontbreekt alleen bij bepaalde inkomensgroepen.
- MNAR (Missing Not At Random):
  - Ontbrekende waarden hangen samen met de ontbrekende waarde zelf.
  - Voorbeeld: Mensen die hun gewicht niet rapporteren omdat ze ontevreden zijn.

Strategieën om Ontbrekende Waarden te Behandelen

- Verwijderen *(niet onze voorkeur*!)*:
  - Rij of kolom verwijderen met dropna():
    - df.dropna() verwijdert rijen met ontbrekende waarden.
    - df.dropna(axis=1) verwijdert kolommen met ontbrekende waarden.

** Enkel indien de record onvoldoende informatie bijdraagt, bijvoorbeeld enkel een naam, een datum,... met voor de rest geen nuttige info. Ook afhankelijk van de grootte van de dataset en de tijd die je eraan kan besteden...*

Strategieën om Ontbrekende Waarden te Behandelen

- Gebruik van fillna() om ontbrekende waarden te vervangen.
  - Gemiddelde, Mediaan, Modus:
    - df['kolom'].fillna(df['kolom'].mean(), inplace=True)
    - Geschikt voor numerieke gegevens.
  - Via sci-kit learn:
    - from sklearn.impute import SimpleImputer
      imputer = SimpleImputer(strategy='mean')
      df_imputed = pd.DataFrame(imputer.fit_transform(df), columns=df.columns)
    - KNNImputer van scikit-learn.
  - Forward/Backward Filling:
    - Geschikt voor tijdreeksen.
    - df.fillna(method='ffill') of df.fillna(method='bfill').

Python.exposed Learn Python to Automate, Analyze, Accelerate.

Afwegingen bij Imputatiemethoden

- Verwijderen:
  - Voordeel: Simpel en snel.
  - Nadeel: Verlies van data kan leiden tot minder nauwkeurige modellen.
- Imputatie met Gemiddelde/Mediaan/Modus:
  - Voordeel: Behoudt alle rijen.
  - Nadeel: Verstoort mogelijk de data-verdeling.
- K-NN Imputatie:
  - Voordeel: Houdt rekening met de relatie tussen gegevens.
  - Nadeel: Kan rekenintensief zijn.
- Forward/Backward Filling:
  - Voordeel: Goed voor continue tijdreeksen.
  - Nadeel: Kan trends in de data vervormen.

python.exposed
Learn Python to Automate, Analyze, Accelerate.

Schalen en Normaliseren van Data

Schalen en Normaliseren van Data

- Wat is schalen?
  - Het aanpassen van de grootte van gegevens zodat ze binnen een bepaald bereik vallen.
  - Voorbeeld: waarden transformeren naar een bereik van 0 tot 1.
- Wat is normaliseren?
  - Het aanpassen van de verdeling van gegevens, zodat deze gemiddeld 0 en een standaardafwijking van 1 hebben.
  - Voorbeeld: Z-score standaardisatie.
- Waarom is dit belangrijk?
  - Helpt om modellen nauwkeuriger te maken.
  - Nodig voor modellen die afhankelijk zijn van afstanden (zoals k-NN en SVM).

Python.exposed Learn Python to Automate, Analyze, Accelerate.

Schalen en Normaliseren van Data

- Veelgebruikte Methoden voor Schalen en Normaliseren

- Min-Max Schaling:
  - Formule: $X_{scaled} = \frac{X - X_{min}}{X_{max} - X_{min}}$

- Resulteert in waarden tussen 0 en 1.
- Gebruik in Python:

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()

scaled_data = scaler.fit_transform(data)
```

Schalen en Normaliseren van Data

- Veelgebruikte Methoden voor Schalen en Normaliseren

- Z-Score Standaardisatie:
  - Formule: $Z = \frac{X - \mu}{\sigma}$
- Resulteert in waarden tussen 0 en 1.
- Gebruik in Python:

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

normalized_data = scaler.fit_transform(data)
```


Schalen en Normaliseren van Data

- Waarom Schalen en Normaliseren?
  - Veel machine learning modellen zijn gevoelig voor de schaal van de data.
- Afstandsgebaseerde modellen:
  - Algoritmen zoals k-NN en SVM berekenen afstanden tussen punten.
- Ongeschaalde data kan leiden tot vertekening in modelresultaten.
- Snellere convergentie:
  - Schaling helpt gradient-gebaseerde algoritmen sneller te trainen.
- Voorbeelden:
  - Bij gebruik van k-NN kan een ongewoon grote variabele de afstandsberekeningen domineren.

Python.exposed
Learn Python to Automate, Analyze, Accelerate.

Wanneer Min-Max Schaling gebruiken?

- Als je wilt dat je data binnen een specifiek bereik valt, bijvoorbeeld tussen 0 en 1. Dit is handig als je met algoritmen werkt die niet goed omgaan met grote variaties in schaal.
- Algoritmen zoals neurale netwerken werken vaak beter met inputwaarden tussen 0 en 1.
- Min-Max schaling is *gevoeliger voor uitschieters* (extreme waarden). Als de dataset geen significante uitschieters heeft, kan dit een geschikte keuze zijn.
- Min-Max schaling kan nuttig zijn voor visualisaties, omdat het de data transformeert naar een uniforme schaal, waardoor kenmerken beter vergelijkbaar zijn.
- Het behoudt de onderlinge verhoudingen van de data, wat betekent dat het lineaire relaties niet verandert.

Wanneer Z-Score Standaardisatie gebruiken?

- Als je dataset uitschieters bevat, is Z-score standaardisatie vaak beter omdat het minder gevoelig is voor extreme waarden. Het zet de data om in een vorm waarbij uitschieters duidelijker zichtbaar blijven, zonder de schaal van de rest van de data sterk te beïnvloeden.
- Als je wilt dat de data een normale verdeling heeft, of als het model dit veronderstelt (bijvoorbeeld in lineaire regressie)
- Afstandsgebaseerde Modellen zoals k-NN, SVM, en PCA werken vaak beter met gestandaardiseerde data, omdat de schaal van de kenmerken gelijk getrokken wordt.
- Als de kenmerken verschillende eenheden hebben (bijvoorbeeld lengte in centimeters en gewicht in kilogrammen), kan Z-score standaardisatie helpen om ze vergelijkbaar te maken.

Python exposed

Schalen en Normaliseren van Data

- Samengevat:
  - Gebruik Min-Max schaling als je vaste bereiken wilt en de data geen uitschieters heeft.
  - Gebruik Z-score standaardisatie als je rekening wilt houden met de verdeling van de data en robuuster wilt zijn tegen uitschieters.

python.exposed
Learn python to automate, analyze, accelerate.

Dubbele data en aggregeren van waarden

Dubbele waarden

- Python zorgt dat we dubbele waarden snel en eenvoudig kunnen opsporen en verwijderen

```
# Dubbele rijen identificeren en verwijderen
df_uniek = df.drop_duplicates()
```

```
Origineel DataFrame:
    Naam  Leeftijd       Stad
0    Jan        25  Amsterdam
1   Piet        30  Rotterdam
2  Klaas        22    Utrecht
3  Maria        29  Amsterdam
4   Anne        32   Den Haag
5    Jan        25  Amsterdam
6  Klaas        22    Utrecht
7  Maria        29  Amsterdam
8    Els        27  Eindhoven
9   Piet        30  Rotterdam

DataFrame zonder dubbele rijen:
    Naam  Leeftijd       Stad
0    Jan        25  Amsterdam
1   Piet        30  Rotterdam
2  Klaas        22    Utrecht
3  Maria        29  Amsterdam
4   Anne        32   Den Haag
8    Els        27  Eindhoven
```


SYNTRA ANTWERPEN & VLAAMS-BRABANT

Aggregatie van waarden

- Soms vertegenwoordigen dubbele rijen meerdere vermeldingen voor dezelfde entiteit.
- In plaats van de dubbele rijen te verwijderen, kun je ze samenvoegen en informatie samenvatten.

```
# Voorbeelddata
data = {
    'customer_id': [101, 102, 101, 103, 102],
    'product_id': ['A', 'B', 'A', 'C', 'B'],
    'quantity_sold': [5, 3, 2, 1, 4]
}

df = pd.DataFrame(data)
print(df, end="\n\n")
# Samenvoegen van dubbele records op basis van 'customer_id' en 'product_id'
merged_df = df.groupby(['customer_id', 'product_id']).sum().reset_index()
```

```
   customer_id product_id  quantity_sold
0          101          A              5
1          102          B              3
2          101          A              2
3          103          C              1
4          102          B              4

   customer_id product_id  quantity_sold
0          101          A              7
1          102          B              7
2          103          C              1
```


SYNTRA ANTWERPEN & VLAAMS-BRABANT

Parsen van datums

Parsen van Datums

- Wat is date-time parsing?
  - Het omzetten van tekstuele datum-tijd waarden naar een datetime-formaat dat bruikbaar is voor analyse.
- Waarom?
  - Datums bevatten waardevolle informatie zoals seizoensgebonden trends, maandelijkse patronen, enz.
  - Helpt bij het uitvoeren van tijdreeksenanalyse en bij het verkrijgen van inzicht in tijdsgebonden gegevens.

Datum-Tijd Representaties in Datasets

- Datums worden vaak als string (bijv. "2023-10-17") of als Unix-tijdstempel (bijv. 1609459200) opgeslagen in datasets.
  - Verschillende formaten:
    - "DD-MM-YYYY"
    - "YYYY/MM/DD"
    - "MM/DD/YYYY"
    - enz
- Variatie in formaten maakt directe analyse moeilijk.
- Gebruik daarom Pandas om datums te parsen en te standaardiseren.

SYNTRA ANTWERPEN & VLAAMS-BRABANT

Datum Parsing met Pandas

- Pandas functies voor datum parsing:
  - pd.to_datetime():
    - Converteert een kolom of reeks naar een datetime-formaat.
    - Gebruik: df['datum'] = pd.to_datetime(df['datum'], format='%Y-%m-%d')
    - Herkent automatisch veelgebruikte formaten.
  - parse_dates bij het inladen van data:
    - df = pd.read_csv('data.csv', parse_dates=['datum'])

SYNTRA ANTWERPEN & VLAAMS-BRABANT

Extraheren van Datum- en Tijdcomponenten

- Belangrijke componenten:
  - Dag, Maand, Jaar:
    - df['datum'].dt.day,
    - df['datum'].dt.month,
    - df['datum'].dt.year
  - Uur, Minuut, Seconde:
    - df['datum'].dt.hour,
    - df['datum'].dt.minute,
    - df['datum'].dt.second
  - Weekdag: df['datum'].dt.dayofweek (0 = Maandag)
  - Weeknummer: df['datum'].dt.isocalendar().week
  - Voorbeeld:
    - df['jaar'] = df['datum'].dt.year
    - df['maand'] = df['datum'].dt.month
    - df['dag'] = df['datum'].dt.day

SYNTRA ANTWERPEN & VLAAMS-BRABANT

Omgaan met Tijdzones en Zomertijd

- Tijdzones in Pandas:
  - tz_localize(): Zet een datumkolom om naar een specifieke tijdzone.
  - Gebruik: df['datum'] = df['datum'].dt.tz_localize('Europe/Amsterdam')
  - tz_convert(): Converteert tijd van de ene tijdzone naar de andere.
  - Gebruik: df['datum'] = df['datum'].dt.tz_convert('UTC')
- Zomertijd (Daylight Savings Time):
  - Belangrijk om te corrigeren voor afwijkingen in tijd.
  - Pandas en de pytz-bibliotheek kunnen helpen om automatisch te schakelen tussen standaardtijd en zomertijd.
  - Let op: Controleer of je dataset tijdzones correct ondersteunt om fouten te voorkomen.

SYNTRA ANTWERPEN & VLAAMS-BRABANT

Python.exposed Learn Python to Automate, Analyze & Accelerate.

Standaardiseren van datums

```python
import dateparser

# Voorbeeld DataFrame met verschillende datumformaten
data = {
    'naam': ['Jan', 'Els', 'Piet', 'Jona', 'Dieter', 'Jason'],
    'datum_aankoop': ['2023-01-01', '01/02/2023', 'March 5, 2023', 'April 10th, 2023', '2023/05/15', 'next Monday']
}

df = pd.DataFrame(data)

# Functie om datum te parseren
df['datum_aankoop'] = df['datum_aankoop'].apply(lambda x: dateparser.parse(x))

print("Gecorrigeerde datums:")
print(df)
```

```
Gecorrigeerde datums:
     naam datum_aankoop
0     Jan    2023-01-01
1     Els    2023-01-02
2    Piet    2023-03-05
3    Jona    2023-04-10
4  Dieter    2023-05-15
5   Jason           NaT
```


python.exposed
Learn Python to Automate, Analyze, Accelerate.

Character Encoding

Karaktercodering

- Een manier om tekens (letters, cijfers, symbolen) om te zetten in binaire getallen die computers begrijpen.
- Elke tekenset heeft een bijbehorende code (bijv. A = 65 in ASCII).
- Karaktercodering zorgt ervoor dat tekst correct wordt weergegeven op verschillende apparaten en software.
- Helpt bij het verwerken en analyseren van tekstdata in datasets.

Soorten Karaktercoderingen

- ASCII (American Standard Code for Information Interchange):
  - Voor basis Engelse tekens (A-Z, a-z, cijfers, en enkele speciale tekens).
  - Beperking: Kan geen niet-Engelse tekens opslaan (bijv. é, ü, ç).
  - Gebruikt 7 bits per teken.
- UTF-8 (Unicode Transformation Format - 8 bit):
  - Meestgebruikte codering: Ondersteunt wereldwijd alle tekens.
  - Compact voor Engelse tekens (1 byte), maar kan meerdere bytes gebruiken voor complexere tekens.
  - Geschikt voor internationale applicaties en webpagina's.
- Overige Coderingen:
  - ISO-8859-1 (Latin-1): Ondersteunt Europese tekens.
  - UTF-16: voor talen met veel speciale tekens, maar gebruikt meer geheugen.

Python.exposed
Learn python to automate, analyze, accelerate.

Belang van Karaktercodering in datasets

- Waarom is karaktercodering belangrijk in datasets?
- Correcte Weergave: Tekst kan onleesbaar worden (bijv. "garbled text") als verkeerde codering wordt gebruikt.
- Internationale Data: Data van verschillende bronnen kan verschillende coderingen hebben.
- Bestandssamenvoeging: Bij het combineren van bestanden is het essentieel dat de codering consistent is.
- Veelvoorkomende Problemen:
- Vervormde tekens (bijv. Ã© in plaats van é).
- UnicodeDecodeErrors bij het laden van bestanden in Python.

SYNTRA ANTWERPEN & VLAAMS-BRABANT

Karaktercodering in Python en Pandas

- Bestanden inladen met specifieke codering:
  - pd.read_csv('data.csv', encoding='utf-8') voor UTF-8 bestanden.
  - Andere veelgebruikte opties: ISO-8859-1, utf-16.
- Veelvoorkomende Encodering Fouten:
  - UnicodeDecodeError: Treedt op als de opgegeven codering niet overeenkomt met de werkelijke codering van het bestand.
  - UnicodeEncodeError: Als een karakter niet kan worden omgezet naar de opgegeven codering.

SYNTRA ANTWERPEN & VLAAMS-BRABANT

Karaktercodering in Python en Pandas

- Oplossingen voor de veelvoorkomende fouten:
  - Gebruik errors='replace' om onbekende tekens te vervangen:
  - df = pd.read_csv('data.csv', encoding='utf-8', errors='replace')
- Gebruik chardet om de juiste codering automatisch te detecteren:
  - import chardet
  - with open('data.csv', 'rb') as f:
  - result = chardet.detect(f.read(10000))
  - print(result['encoding'])

Learn Python to Automate, Analyze, Accelerate.
Python.exposed

Karaktercodering in Python en Pandas

- Oplossingen voor de veelvoorkomende fouten:
  - Gebruik errors='replace' om onbekende tekens te vervangen:
  - df = pd.read_csv('data.csv', encoding='utf-8', errors='replace')
- Gebruik chardet om de juiste codering automatisch te detecteren:
  - import chardet
  - with open('data.csv', 'rb') as f:
  - result = chardet.detect(f.read(10000))
  - print(result['encoding'])

Wanneer welke Karaktercodering gebruiken?

- UTF-8:
  - Voor data uit internationale bronnen, webdata, en moderne applicaties.
  - Voordeel: Flexibel en universeel.
- ASCII:
  - Voor simpele datasets zonder speciale tekens.
  - Voordeel: Efficiënt qua opslagruimte.
  - Nadeel: Beperkt in tekens.
- ISO-8859-1 (Latin-1):
  - Voor data met veel Europese tekens.
  - Voordeel: Eenvoudig te gebruiken voor oudere Europese data.
  - Nadeel: Kan niet alle Unicode-tekens verwerken.

python.exposed
Learn Python to Automate, Analyze, Accelerate.

50

Wanneer welke Karaktercodering gebruiken?

- UTF-8:
  - Voor data uit internationale bronnen, webdata, en moderne applicaties.
  - Voordeel: Flexibel en universeel.
- ASCII:
  - Voor simpele datasets zonder speciale tekens.
  - Voordeel: Efficiënt qua opslagruimte.
  - Nadeel: Beperkt in tekens.
- ISO-8859-1 (Latin-1):
  - Voor data met veel Europese tekens.
  - Voordeel: Eenvoudig te gebruiken voor oudere Europese data.
  - Nadeel: Kan niet alle Unicode-tekens verwerken.

Maak werk van je toekomst

SYNTRA ANTWERPEN & VLAAMS-BRABANT

Inconsistente data

SYNTRA ANTWERPEN & VLAAMS-BRABANT

Inconsistente gegevensinvoer

- Variatie in de manier waarop dezelfde informatie wordt vastgelegd.
  - Voorbeeld: "Amsterdam", "amsterdam", "A'dam".
- Kan leiden tot onnauwkeurige analyses en dubbele gegevens.
- Verstoort data-integratie bij het combineren van meerdere bronnen.
- Beïnvloedt de prestaties van machine learning-modellen door ruis in de data.

SYNTRA ANTWERPEN & VLAAMS-BRABANT

Inconsistente gegevensinvoer

- Verschillende Formaten:
  - Datumnotaties: "2024-10-17", "17/10/2024", "October 17, 2024".
- Hoofdlettergevoeligheid (Case Sensitivity):
  - Verschillen in hoofdlettergebruik zoals "Customer A" vs. "customer a".
- Typfouten en Spelfouten:
  - Bijv. "Janssen" vs. "Jansen".
  - Kunnen ontstaan door handmatige invoer.
- Gebruik van Afkortingen:
  - Bijvoorbeeld "NYC" vs. "New York City".

SYNTRA ANTWERPEN & VLAAMS-BRABANT

Detecteren en Opschonen van Inconsistente Data

- String Matching:
  - Gebruik functies zoals .str.casefold() om hoofdlettergevoeligheid te negeren:
    - df['kolom'] = df['kolom'].str.casefold()
  - Vervangen van Ongewenste Tekens:
    - Gebruik .str.replace() om specifieke tekens te vervangen:
      - df['adres'] = df['adres'].str.replace('-', ' ')
- Regular Expressions (Reguliere Expressies):
  - Standaardiseren van patronen in tekst.
  - Voorbeeld: Alles omzetten naar een standaard formaat met str.replace() en regex.
    - df['telefoonnummer'] = df['telefoonnummer'].str.replace(r'\D', '') (verwijdert niet-numerieke tekens).
-

SYNTRA ANTWERPEN & VLAAMS-BRABANT

IT python.exposed learn Python to automate, analyze, accelerate.

Detecteren en Opschonen van Inconsistente Data

```
Origineel DataFrame:
     naam geslacht leeftijd  datum_aankoop
0    Jan         M       25     2023-01-01
1   maria        F   thirty     01/02/2023
2     Els        F       27  March 5, 2023
3  PETER         M       40     2023/04/10
4    AnNe        f
```

```python
# Verwijder leidende en afsluitende spaties en maak alle namen met kleine letters
df['naam'] = df['naam'].str.strip().str.capitalize()

# Gebruik mapping om 'geslacht' te standaardiseren
value_mapping = {'M': 'Male', 'F': 'Female', 'f': 'Female'}
df['geslacht'] = df['geslacht'].map(value_mapping)

# Probeer leeftijden om te zetten naar numerieke waarden,
# gebruik 'errors' om niet-numerieke waarden om te zetten naar NaN
df['leeftijd'] = pd.to_numeric(df['leeftijd'], errors='coerce')

# Datumformaten standaardiseren
df['datum_aankoop'] = pd.to_datetime(df['datum_aankoop'], errors='coerce')

print("\nGecorrigeerd DataFrame:")
print(df)
```

```
Gecorrigeerd DataFrame:
    naam geslacht  leeftijd datum_aankoop
0    Jan      NaN      25.0    2023-01-01
1  Maria      NaN       NaN           NaT
2    Els      NaN      27.0           NaT
3  Peter      NaN      40.0           NaT
4   Anne      NaN       NaN    2023-05-15
```


python.exposed Learn Python to Automate, Create, Accelerate.

Detecteren en Opschonen van Inconsistente Data

- str.strip() en str.capitalize(): Verwijdert spaties en zorgt dat alleen de eerste letter van elke naam een hoofdletter is.
- map(): Gebruikt een mapping om vb verkorte aanduidingen van 'geslacht' om te zetten naar volledige woorden ('Male' en 'Female').
- pd.to_numeric(): Zet de waarden in de kolom 'leeftijd' om naar numerieke waarden. Niet-numerieke waarden (zoals 'thirty') worden omgezet naar NaN.
- pd.to_datetime(): Zet de verschillende datumformaten om naar een gestandaardiseerd datetime-formaat.

Hulpmiddelen voor Gegevensopschoning

- FuzzyWuzzy
  - Een Python-bibliotheek voor fuzzy string matching.
  - Meet de gelijkenis tussen tekststrings.
- Toepassingen:
  - Identificeer vergelijkbare, maar niet identieke namen of adressen.
  - Bijv. "Jansen" vs. "Janssen" kan worden herkend als 90% overeenkomst.

python.exposed Learn Python to Automate, Analyze, Accelerate.

Hulpmiddelen voor Gegevensopschoning

- Voorbeeld Gebruik:
  - from fuzzywuzzy import fuzz
  - fuzz.ratio('Jansen', 'Janssen') geeft een overeenkomstscore van 90.
- Batch-matching met process voor grotere datasets:
  - from fuzzywuzzy import process
  - process.extract('Janssen', lijst_met_namen, limit=3)

Gegevensopschoning met Pandas en FuzzyWuzzy

- Stap 1: Hoofdletters Uniform Maken:
  - df['kolom'] = df['kolom'].str.lower()
- Stap 2: Whitespace Verwijderen:
  - df['kolom'] = df['kolom'].str.strip()
- Stap 3: Gebruik van Reguliere Expressies:
  - Vervang ongewenste tekens met str.replace():
    - df['postcode'] = df['postcode'].str.replace(r'\s+', '') (verwijdert alle spaties).
- Stap 4: Fuzzy Matching Toepassen:
  - Identificeer en corrigeer vergelijkbare namen met FuzzyWuzzy.
  - Gebruik fuzz.ratio om potentiële overeenkomsten te vinden en te corrigeren.

python.exposed
Learn Python to automate, analyze, accelerate.

Samenvoegen van data

Samenvoegen van data

- Voor analyses zijn vaak gegevens uit verschillende bronnen nodig (bijv. klantgegevens en transactiegegevens).
- Samenvoegen van datasets helpt bij het creëren van een completer beeld van de data.

- Datasets combineren in Pandas:
  - Concatenatie: Rijen of kolommen samenvoegen.

(# Samenvoegen langs kolommen)

```
df_combined = pd.concat([df1, df2], axis=1)
```


python.exposed Learn Python to Automate, Analyze, Accelerate.

Samenvoegen van data

- Datasets combineren in Pandas:
  - Merges: Datasets samenvoegen op basis van gemeenschappelijke sleutels.

(# Inner join op 'customer_id')

```
df_merged = pd.merge(df1, df2, on='customer_id', how='inner')
```

Verschillende soorten merges:
Inner join: Behoudt alleen overeenkomende rijen.
Outer join: Behoudt alle rijen, vult ontbrekende waarden in met NaN.
Left/Right join: Behoudt alle rijen van de linker/rechter dataset.

Goede praktijken

Best practices

- Bewaar Ruwe Data Apart – altijd!
  - Bewaar een kopie van de originele, ruwe data voordat je wijzigingen aanbrengt.
  - Geef het bestand een naam zoals “-RAW” om het gemakkelijk te herkennen.
  - Hiermee kun je altijd terug naar de oorspronkelijke data als dat nodig is.
- Documenteer Je Code
  - Voeg opmerkingen toe om uit te leggen wat elke stap van de schoonmaak doet.
  - Noteer eventuele aannames die je hebt gemaakt.
  - Dit maakt je code begrijpelijker voor jezelf en anderen.

Best practices

- Let op onvoorziene gevolgen
  - Blijf de data verkennen tijdens het schoonmaken:
  - Zorg ervoor dat de schoonmaak geen grote veranderingen in de data veroorzaakt.
  - Voorkom het introduceren van vooroordelen.
  - Herhaal verkenning van de data om zeker te zijn dat alles klopt.
- Houd een “schoonmaaklogboek” bij. Noteer details zoals:
  - Datum van de actie
  - Specifieke schoonmaakacties
  - Eventuele problemen of bijzonderheden

python.exposed
Learn Python to Automate, Analyze, Accelerate.

Best practices

- Schrijf Herbruikbare Functies
  - Maak functies voor veelvoorkomende schoonmaaktaken.
  - Dit helpt je om dezelfde schoonmaakstappen toe te passen op meerdere datasets.
  - Vooral handig voor bedrijfsspecifieke aanpassingen (zoals afkortingen).
- Gebruik een checklist

Dank u
voor uw aandacht

python.exposed
Learn Python to Automate, Analyze, Accelerate.